

Aprendizaje No Supervisado

Agenda

Contenido de la sesión

- **1.** Introducción al Aprendizaje No Supervisado
- **2.** Agrupamiento (Clustering)
 - K-Means, DBSCAN
 - Métricas de evaluación
- **3.** PCA — Análisis de Componentes Principales

SECCIÓN 1

Introducción al Aprendizaje No Supervisado

Definición, tipos de tareas y métricas de evaluación

Aprendizaje No Supervisado

Definición y diferencias con otros paradigmas

Supervisado

- Los datos tienen **etiquetas** conocidas (y)
- El modelo aprende la función $f(x) \rightarrow y$
- Ejemplos: clasificación, regresión

Semi-supervisado

- Mezcla de datos etiquetados y **sin etiquetar**
- Aprovecha la estructura latente del conjunto

No supervisado

- Los datos **no tienen etiquetas**
- El modelo descubre patrones, estructura y agrupaciones por sí solo
- Objetivo: aprender la **distribución subyacente** de los datos

El modelo no recibe retroalimentación externa sobre si sus predicciones son correctas (IBM, s.f.; Chust, s.f.).

Tipos de Tareas No Supervisadas

Categoría	Objetivo	Ejemplos
Clustering	Agrupar observaciones similares	K-Means, DBSCAN, Jerárquico
Reducción de dimensionalidad	Comprimir información preservando varianza	PCA, t-SNE, UMAP
Modelos generativos	Aprender la distribución para generar nuevos datos	VAE, GAN, modelos de mezcla
Detección de anomalías	Identificar puntos atípicos o inusuales	Isolation Forest, One-Class SVM
Sistemas de recomendación	Descubrir relaciones latentes entre usuarios e ítems	Filtrado colaborativo

Métricas de Evaluación Sin Etiquetas

Intrínsecas (sin ground truth)

Miden la calidad interna de los agrupamientos:

- **Silhouette Score** — cohesión vs. separación
- **Davies-Bouldin Index** — relación entre dispersión y distancia entre clusters
- **Calinski-Harabasz Index** — varianza inter-cluster vs. intra-cluster

Extrínsecas (con etiquetas de referencia)

Comparan los clusters con etiquetas conocidas (solo para evaluación, no para entrenamiento):

- **Rand Index (RI / ARI)**
- **Mutual Information (NMI)**
- **Homogeneity & Completeness**

En producción las métricas intrínsecas son las únicas disponibles. Las extrínsecas solo se usan en benchmarking (Kashnitsky, s.f.).

SECCIÓN 2

Agrupamiento (Clustering)

K-Means, Jerárquico, DBSCAN y evaluación

K-Means

Algoritmo, centroides y convergencia

Pasos del algoritmo

1. **Inicializar** K centroides aleatorios (o con K-Means++)
2. **Asignar** cada punto al centroide más cercano (distancia euclidiana)
3. **Recalcular** el centroide de cada cluster como la media de sus puntos
4. **Repetir** pasos 2–3 hasta convergencia (sin cambios en las asignaciones)

Función objetivo (inercia):

$$J = \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2$$

Minimiza la suma de distancias cuadradas al centroide.

Limitaciones:

- Sensible a la inicialización
- Asume clusters **esféricos** y de **tamaño similar**
- Requiere definir K a priori

K-Means++

Inicialización inteligente de centroides

El método estándar de inicialización aleatoria puede converger a **mínimos locales** subóptimos.

K-Means++ (Arthur & Vassilvitskii, 2007)

1. Elegir el primer centroide **aleatoriamente** de los datos
2. Para cada punto restante, calcular su distancia al centroide más cercano ya elegido: **$D(x)$**
3. Seleccionar el siguiente centroide con probabilidad proporcional a **$D(x)^2$**
4. Repetir hasta tener K centroides

K-Means++ garantiza una solución $O(\log K)$ -competitiva en esperanza respecto al óptimo global. Reduce drásticamente iteraciones necesarias y mejora la calidad del clustering final (Kashnitsky, s.f.).

Selección del Número de Clusters

Elbow Method

Grafica la **inercia** (WCSS) en función de K.

- La inercia es la suma de las distancias al cuadrado entre los puntos y el centroide.
- La inercia disminuye monotónicamente al aumentar K
- El "codo" marca el punto de **rendimientos decrecientes**
- K óptimo = punto de inflexión de la curva

Silhouette Score

Mide la cohesión y separación.

Para cada punto i :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- **a(i)**: distancia media intra-cluster
- **b(i)**: distancia media al cluster vecino más cercano
- Rango: **[-1, 1]** — cuanto mayor, mejor separación

Combinar ambas métricas evita elegir K erróneamente cuando el codo no es claro (Kashnitsky, s.f.).

DBSCAN

Clustering basado en densidad

Density-Based Spatial Clustering of Applications with Noise

Parámetros clave

- **ϵ (eps):** radio de vecindad
- **MinPts:** número mínimo de puntos para formar una región densa

Clasificación de puntos

Tipo	Condición
Core point	Tiene $\geq$ MinPts vecinos dentro de radio ε
Border point	Vecino de un core point pero con $<$ MinPts vecinos propios
Noise (outlier)	No es core ni border point

Ventaja clave: detecta clusters de forma arbitraria y etiqueta automáticamente ruido y outliers sin necesitar K

Cálculo de ε con Distancia K

Se fija como $\text{MinPts} = 4$ (o doble de las dimensiones)

1. Para cada punto del dataset, calcular la distancia a su 4to vecino más cercano.
2. Ordenar las distancias de menor a mayor.
3. Graficar los puntos en el eje X (ordenados) y su distancia en el eje Y.

Evaluación de Clustering

Silhouette, Davies-Bouldin y Calinski-Harabasz

Métrica	Fórmula	Interpretación	Óptimo
Silhouette	$(b-a)/\max(a,b)$	Cohesión vs. separación por punto	Máximo $\rightarrow$ 1
Davies-Bouldin	Media de $\max(R_{i,j})$	Razón dispersión intra / distancia inter	Mínimo $\rightarrow$ 0
Calinski-Harabasz	$\text{Tr}(B_k)/\text{Tr}(W_k) \times (n-k)/(k-1)$	Varianza entre clusters vs. dentro	Máximo

Cuándo usar cada una

- **Silhouette:** comparar distintos K o algoritmos; interpretable intuitivamente
- **Davies-Bouldin:** penaliza clusters compactos pero cercanos entre sí
- **Calinski-Harabasz:** rápida de calcular; útil para grandes datasets

(Kashnitsky, s.f.)

SECCIÓN 3

PCA

Análisis de Componentes Principales

Fundamentos Matemáticos de PCA

Varianza, covarianza, eigenvectores y eigenvalores

Idea central

PCA encuentra las direcciones de **máxima varianza** en los datos proyectando a un nuevo espacio ortogonal.

Pasos matemáticos:

1. Centrar los datos: $\tilde{X} = X - \mu$
2. Calcular la **matriz de covarianza**: $\Sigma = (1/n) \tilde{X}^T \tilde{X}$
3. Calcular **eigenvectores** (componentes principales) y **eigenvalores** de Σ
4. Ordenar por eigenvalor descendente
5. Proyectar: $Z = \tilde{X} \cdot W_k$

Interpretación:

- **Eigenvalores** → varianza capturada por cada componente
- **Eigenvectores** → dirección de cada componente principal
- Componentes son **ortogonales** entre sí (sin correlación)

PCA es equivalente a la SVD de la matriz de datos centrada (UC Berkeley CS 189, s.f.).

Reducción de Dimensionalidad con PCA

Varianza explicada y selección de componentes

Varianza explicada acumulada

$$\text{VE acumulada}(k) = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^p \lambda_i}$$

donde λ_i son los eigenvalores ordenados de mayor a menor.

Criterios de selección de K:

- **Umbral de varianza:** retener K que explique $\geq 95\%$ (o 90%) de la varianza total
- **Scree plot:** buscar el "codo" en la curva de eigenvalores
- **Kaiser rule:** retener componentes con eigenvalor > 1

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95) # 95% varianza
X_reduced = pca.fit_transform(X_scaled)

print(pca.explained_variance_ratio_)
print(pca.n_components_)
```

(Kashnitsky, s.f.; DataCamp, s.f.)

Visualización con PCA

Proyección de datos a 2D y 3D

PCA permite proyectar datos de alta dimensión a 2 o 3 componentes para **visualización exploratoria**.

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca2d = PCA(n_components=2)
X_2d = pca2d.fit_transform(X_scaled)

plt.scatter(X_2d[:, 0], X_2d[:, 1], c=labels, cmap='tab10')
plt.xlabel(f'PC1 ({pca2d.explained_variance_ratio_[0]*100:.1f}%)')
plt.ylabel(f'PC2 ({pca2d.explained_variance_ratio_[1]*100:.1f}%)')
plt.title('Proyección PCA 2D')
plt.show()
```

Etiquetar los ejes con el porcentaje de varianza explicada es buena práctica: permite evaluar cuánta información se pierde en la proyección. Para datos muy no lineales, t-SNE o UMAP producen mejores visualizaciones.

Preprocesamiento antes de PCA

Normalización y estandarización

PCA es sensible a la escala de las variables. Sin estandarización, las variables con mayor magnitud dominarán las primeras componentes, independientemente de su relevancia real.

StandardScaler (recomendado para PCA)

$$z = \frac{x - \mu}{\sigma}$$

- Media 0, desviación estándar 1
- Preserva la forma de la distribución

MinMaxScaler

$$z = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- Rango [0, 1]
- Sensible a outliers

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components=0.95))
])

X_transformed = pipeline.fit_transform(X)
```

Referencias

- 4Geeks. (s.f.). *Aprendizaje no supervisado*. 4Geeks Academy.
<https://4geeks.com/es/lesson/aprendizaje-no-supervisado>
- AI Future School. (s.f.). *Unsupervised learning techniques explained*. <https://www.ai-futureschool.com/en/programming/unsupervised-learning-techniques-explained.php>
- Chust, A. (s.f.). *Aprendizaje no supervisado*. <https://adrianchust.com/aprendizaje-no-supervisado/>
- DataCamp. (s.f.). *Unsupervised learning in Python* [Curso en línea].
<https://www.datacamp.com/courses/unsupervised-learning-in-python>

- IBM. (s.f.). *¿Qué es el aprendizaje no supervisado?* IBM Think.
<https://www.ibm.com/think/topics/unsupervised-learning>
- Kashnitsky, Y. (s.f.). *Topic 7: Unsupervised learning — PCA and clustering* [Notebook]. Kaggle.
<https://www.kaggle.com/code/kashnitsky/topic-7-unsupervised-learning-pca-and-clustering>
- MICROCHIPOTLE. (s.f.). *Aprendizaje no supervisado en machine learning: técnicas y aplicaciones*. <https://microchipotle.com/aprendizaje-no-supervisado-en-machine-learning-tecnicas-y-aplicaciones/>
- UC Berkeley. (s.f.). *CS 189/289A: Introduction to Machine Learning* [Notas de curso].
<https://people.eecs.berkeley.edu/~jrs/189/>